

MCP: Primitives, Servers and Scopes

The three things a server exposes, the three things a client exposes back, and where the configuration lives.

MCP is a **standard connector**. It does not make Claude cleverer. It makes integrations reusable.

Host, client, server

01

Host

The application. Claude Code, the desktop app, your own agent.

02

Client

One connection per server, held by the host.

03

Server

A process or service exposing capabilities.

04

Transport

stdio for local, Streamable HTTP for remote.

ON THE EXAM

One host, many clients, one server each. The host decides which capabilities the model ever sees.

What a server exposes

PRIMITIVE	WHAT IT IS	WHO INVOKES IT
Tools	Actions with typed inputs. Do something.	The model decides
Resources	Read-only data addressed by URI. Know something.	The application or user attaches it
Prompts	Reusable templates with parameters.	The user chooses it

ON THE EXAM

Three primitives, three different controllers. That third column is what most confusion pairs turn on.

Resource or tool?

RESOURCE

Data you want in context

- Read-only, addressed by a URI
- Declares a MIME type
- Pulled in deliberately, not called
- A schema, a config file, a catalogue

TOOL

An action with a result

- Executes logic and may change things
- Typed input schema, model-selected
- Costs a turn and a round trip
- A query, a write, a computation

THE TIEBREAKER

If the agent needs to know it before deciding anything, it is a resource. If it needs to do it, it is a tool.

What resources are quietly good at

- ◆ **Stopping exploratory tool calls.** A catalogue in context beats three probing calls to find out what exists.
- ◆ **Schemas and conventions.** The database structure, the status enum, the naming rules.
- ◆ **Anything stable and referenced repeatedly.** Fetching it once beats fetching it per question.
- ◆ **Not for large volatile data.** Attaching it costs context whether the agent needs it or not.

ON THE EXAM

"The agent makes several calls just to discover what is available" is a resource-shaped problem.

Prompts are the user-facing primitive

- ◆ **Reusable templates with parameters**, published by the server.
- ◆ **In Claude Code they appear as slash commands**, named `/mcp__server__prompt`.
- ◆ **The user chooses them**. The model does not decide to invoke a prompt.
- ◆ **Good for standardising a request** a team makes constantly, with the right structure every time.

ON THE EXAM

A team repeating the same awkward request by hand is a prompt-shaped problem, not a tool-shaped one.

The other direction: what a client exposes

PRIMITIVE	WHAT IT DOES
Sampling	The server asks the host's model for a completion.
Elicitation	The server asks the user for missing input or confirmation.
Roots	The client tells the server which filesystem boundaries it may work within.

ON THE EXAM

MCP is bidirectional. A server is not only answering; it can ask the client for things mid-operation.

Transports

TRANSPORT	WHEN	NOTES
stdio	The server runs as a local process	The default. Works offline. No network involved.
Streamable HTTP	The server is remote	The current standard for hosted servers.
SSE	Legacy remote servers	Older. Existing servers are migrating to HTTP.

ON THE EXAM

Local process means stdio. A URL means HTTP, unless the vendor specifically says SSE.

The three scopes in Claude Code

SCOPE	WHERE IT LIVES	WHO GETS IT
local (default)	Your home config, keyed by project path	Just you, just this project
project	.mcp.json at the project root	Anyone who clones the repository
user	Your home config, global	Just you, in every project

ON THE EXAM

Project scope is the one to know. It is a committed file, so the team gets the same servers automatically.

Three handles once a server is connected

- ◆ **Tools** appear to the model as `mcp__server__tool`.
- ◆ **Prompts** appear to the user as `/mcp__server__prompt`.
- ◆ **Resources** are pulled into a prompt with an `@` mention.
- ◆ **That tool name is what you write in a hook matcher or a permission rule**, so it is worth knowing the shape.

ON THE EXAM

The naming convention is not trivia. It is how you allow or block one specific MCP tool.

Limits worth knowing

- ◆ MCP tool output warns at around 10,000 tokens and is capped at 25,000 by default.
- ◆ Raise it deliberately if a server legitimately returns more, and think about whether it should.
- ◆ Startup and per-call timeouts are configurable, globally or per server.
- ◆ A server returning 40,000 tokens per call is a tool design problem, not a configuration problem.

ON THE EXAM

Raising a cap to fit a bloated response is treating the alarm rather than the fire.

Adopt a server, or build one?

ADOPT

Someone already solved this

- A maintained server for a common system
- Auth, pagination and errors already handled
- You inherit their fixes
- The default answer for GitHub, Slack, Postgres

BUILD

It is your system

- Internal services nobody else has
- Workflows specific to your organisation
- You control the tool granularity
- You own the maintenance forever

THE TIEBREAKER

Adopting is the default for common systems. Building is for your own, and it is a long-term commitment.

Trust and the third-party server

- ◆ A server you install can expose any tools it likes, with any descriptions.
- ◆ Project-scope servers prompt for approval the first time a team member uses them.
- ◆ You cannot modify a vendor's server, so controls go in hooks and permissions.
- ◆ Tool descriptions are instructions. A description from an untrusted server is untrusted input.

ON THE EXAM

Everything from module 6 applies. If the tool is not yours, the enforcement moves up a layer.

Currency check: the protocol is moving

TOPIC	EXAM ANSWER	CURRENT REALITY
Server primitives	Tools, resources, prompts	Unchanged.
Client primitives	Sampling, roots, elicitation	A 2026 release candidate deprecates roots, sampling and logging.
Protocol state	Stateful, with an initialize handshake	The same release candidate proposes making it stateless.

ON THE EXAM

Answer the exam version. The release candidate is a proposal with a long deprecation window, not today's behaviour.

Reading the stem: MCP questions

WHAT THE STEM SAYS	WHAT IT WANTS
"several calls just to find out what exists"	A resource holding the catalogue
"the whole team should get this automatically"	Project scope, committed .mcp.json
"the team types the same request every day"	An MCP prompt, surfaced as a slash command
"the server runs locally as a process"	stdio transport
"we cannot modify the vendor's server"	A hook or permission rule on mcp__server__tool
"the server needs the model to summarise something"	Sampling

ON THE EXAM

These are recall questions wearing scenarios. The vocabulary is the whole test.

Module 7 in five lines

- ◆ Servers expose tools, resources and prompts. Model-driven, application-driven, user-driven.
- ◆ Clients expose sampling, roots and elicitation back. The protocol runs both ways.
- ◆ Resource if the agent needs to know it. Tool if the agent needs to do it.
- ◆ Local is you, project is the repository, user is you everywhere. Project scope is a committed file.
- ◆ MCP standardises the connection. It does not fix a badly designed tool.

ON THE EXAM

That closes Domain 2. Five scenarios, then Domain 3 and Claude Code.

An analytics agent connects to a database through an MCP server. To answer any question it first calls `list_tables`, then calls `describe_table` two or three times to find the right columns, then finally runs the query.

The schema changes roughly twice a year. Engineers report that simple questions take four or five tool calls and that costs are higher than expected.

What is the most effective change?

- ☐ A Merge `list_tables` and `describe_table` into one tool that returns everything at once
- ☐ B Expose the schema as an MCP resource so it is in context before the agent plans a query
- ☐ C Cache the schema in the agent's system prompt and update it manually twice a year
- ☐ D Add the table list to the query tool's description

◆ PAUSE AND DECIDE

What is the most effective change?

A Merge the two tools

Better than three calls, and it is still a call. The agent is discovering at run time what it could have known upfront.

C Paste it in the system prompt

Works until someone forgets the manual update. The server already knows the schema; let it publish it.

B Schema as a resource

Stable, referenced constantly, needed before deciding anything. That is the resource case exactly.

D Put tables in the description

Descriptions are for how to use a tool. A schema in there goes stale and bloats every request.

THE SIGNAL

"Changes twice a year" plus "needed before every query" is the resource signature. Stable, and required upfront.

A platform team has built an MCP server for their internal deployment system. They want every engineer on the team to have it configured automatically when they clone the repository, without each person running a setup command.

A separate server, a personal note-taking tool, should follow one engineer across all their projects.

Which scopes should be used?

- ☐ A Both at user scope, so they are always available
- ☐ B Deployment server at project scope; note-taking server at user scope
- ☐ C Deployment server at local scope; note-taking server at project scope
- ☐ D Both at project scope, so the configuration is version controlled

◆ PAUSE AND DECIDE

Which scopes should be used?

- A

 Both user scope
User scope is per person. Every engineer would have to add the deployment server themselves.
- B

 Project, then user
Project scope ships with the repository. User scope follows the individual across projects.
- C

 Local, then project
Exactly backwards. Local is one person one project; project pushes a personal tool onto the team.
- D

 Both project scope
The team inherits somebody's note-taking server, and it only works inside that repository.

THE SIGNAL

Two requirements, two different axes. "Everyone on this repository" is project. "This person everywhere" is user.

An incident response team uses Claude Code with an MCP server for their observability platform. Every incident, an engineer types out a long request: pull the last hour of errors for a named service, group them by type, correlate with recent deploys, and format the result as an incident summary.

The wording varies between engineers, and the output format is inconsistent.

What is the most appropriate solution?

- ☐ A Add a tool to the server that performs the whole sequence
- ☐ B Publish an MCP prompt taking the service name, so it appears as a slash command
- ☐ C Add the standard wording to CLAUDE.md so everyone uses the same phrasing
- ☐ D Expose the incident summary format as a resource

◆ PAUSE AND DECIDE

What is the most appropriate solution?

- A

 One big tool
Plausible, and it hard-codes an investigation that engineers vary deliberately. It also removes the agent's judgement.
- B

 An MCP prompt
A user-invoked template with a parameter is exactly this shape. One slash command, same structure every time.
- C

 Wording in CLAUDE.md
Context, not a command. Engineers still have to remember to phrase the request that way.
- D

 Format as a resource
Fixes the format inconsistency and none of the typing. Half the problem, and the smaller half.

THE SIGNAL

A human doing the same thing repeatedly wants a prompt. A model doing the same thing repeatedly wants a tool.

A team wants to integrate their agent with a widely used issue tracker. There is a maintained MCP server for it, published by the vendor, with authentication and pagination handled.

An engineer proposes building an in-house server instead, arguing that they only need four operations and the vendor's server exposes thirty tools.

The team has no capacity for ongoing maintenance.

What should you advise?

- ☐ A Build the in-house server. Four focused tools will outperform thirty overlapping ones
- ☐ B Adopt the vendor's server and restrict which of its tools the agent may use
- ☐ C Adopt the vendor's server as-is, since more tools give the agent more options
- ☐ D Build the in-house server and contribute it back to the vendor

◆ PAUSE AND DECIDE

What should you advise?

A Build in-house

The tool-count argument is sound, and the stem says there is no maintenance capacity. That decides it.

C Adopt as-is

Ignores the engineer's valid point. Thirty tools tax every request and worsen selection.

B Adopt and restrict

Keeps the vendor's auth and fixes, and gets the focused toolset by allowing only the four tools needed.

D Build and contribute

All the maintenance burden of A, plus an open-source commitment on top.

THE SIGNAL

The engineer's diagnosis is right and their prescription is wrong. Permissions give you a narrow toolset without owning the server.

A document processing MCP server needs to summarise long documents as part of one of its tools. The team is deciding how the server should obtain those summaries.

They would prefer not to hold a separate API key, manage their own rate limits, or make the summarisation model a configuration option their customers have to set.

Which MCP capability fits?

- ☐ A Elicitation, asking the user to supply a summary
- ☐ B Sampling, requesting a completion from the host's model through the client
- ☐ C Roots, granting the server access to the model's context
- ☐ D A resource containing the document, so the agent summarises it itself

◆ PAUSE AND DECIDE

Which MCP capability fits?

A Elicitation

Elicitation asks the user for input. Nobody wants to be asked to summarise the document themselves.

C Roots

Roots are filesystem boundaries. The description here is invented.

B Sampling

Exactly what sampling is for. The server borrows the host's model, so no key, no rate limits, no model choice.

D A resource

Moves the work into the main conversation, which is what the server was trying to avoid doing inline.

THE SIGNAL

Three constraints, one capability. No key, no rate limits, no model configuration is the definition of sampling.